

Projektuppgift

Frontend-baserad webbutveckling

API Mashup

Andreas Trella



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Andreas Trella, antr2501@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: 01, 2026

Sammanfattning

Denna rapport beskriver utvecklingen av en webbapplikation i form av en så kallad mashup, där data från flera JSON-baserade webbtjänster kombineras och presenteras i ett gemensamt gränssnitt. Syftet med projektet var att skapa en användarvänlig webbapplikation som ger en geografisk överblick över aktuellt väder i olika städer i Skåne. Applikationen riktar sig till en bred målgrupp och har utformats med fokus på enkelhet, tydlighet och responsiv design.

Webbapplikationen hämtar väderdata från OpenWeather API och kombinerar denna information med en interaktiv karta baserad på Leaflet och OpenStreetMap. Genom att använda koordinater från ett API för att visualisera data i ett annat uppfylls kravet på mashup-funktionalitet. Vid sidladdning presenteras väderdata för flera fördefinierade städer, och användaren kan även söka efter valfri plats för att få aktuell väderinformation.

Projektet har byggts på JavaScript, Fetch API och dynamisk DOM-manipulation. En automatiserad arbetsprocess har implementerats med hjälp av Node.js och Vite för att hantera utveckling, byggprocess och publicering. Versionshantering har skett med Git och projektet har publicerats via GitHub Pages.

Vidare har SCSS använts för att strukturera och effektivisera styling, och olika CSS-animationer har implementerats för att förbättra användarupplevelsen, exempelvis en laddningsindikator och interaktiva hover-effekter. Resultatet är en fungerande webbapplikation med ett applikationsliknande beteende där majoriteten av innehållet hämtas och uppdateras dynamiskt.

Projektet visar hur flera tekniker och tidigare kursmoment kan integreras i en större helhet för att skapa en modern, dynamisk och användarcentrerad webbapplikation.

Innehållsförteckning

Sammanfattning	2
1 Inledning	4
1.1 Bakgrund och problemmotivering	4
1.2 Övergripande syfte	4
1.3 Avgränsningar	4
1.4 Konkreta och verifierbara mål	5
1.5 Översikt	5
1.6 Författarens bidrag	5
2 Teori	6
2.1 Mashup och API-integration	6
2.2 JavaScript och AJAX	6
2.3 Leaflet och kartvisualisering	6
2.4 Node.js och automatiserad arbetsprocess	7
2.5 SCSS och CSS-animeringar	7
2.6 Sammanfattning	7
3 Metod	8
3.1 API-integration och mashup	8
3.2 JavaScript och interaktivitet	8
3.3 Styling med SCSS	9
3.4 Arbetsprocess och versionshantering	9
3.5 Implementation av mashup	9
3.6 Testning och validering	9
4 Konstruktion	10
5 Resultat	17
6 Slutsatser	19
Källförteckning	21

1 Inledning

1.1 *Bakgrund och problemmotivering*

I dagens digitaliserade samhälle är webbapplikationer viktiga verktyg för att snabbt få tillgång till information och fatta beslut baserat på aktuell data. För invånare och besökare i Skåne är väderinformation ett återkommande behov, och det finns ofta flera olika webbtjänster som erbjuder sådan data. Informationen är dock ofta spridd över flera plattformar och presenteras inte alltid på ett visuellt tydligt och lättillgängligt sätt. Det innebär att användaren behöver navigera mellan flera källor för att få en fullständig bild av det aktuella vädret.

Mot denna bakgrund syftar projektet till att utveckla en webbapplikation som kombinerar geografiska kartdata med aktuell väderinformation från externa webbtjänster. Genom att integrera data från olika API:er skapas en dynamisk mashup där användaren snabbt kan få en överblick över vädret på en vald plats utan att behöva byta mellan olika tjänster. Webbapplikationen visualiserar väderförhållanden på en interaktiv karta, vilket både förenklar användarupplevelsen och illustrerar hur olika datakällor kan samverka i en modern webbapplikation.

1.2 *Övergripande syfte*

Det övergripande syftet med projektet är att utveckla en enkel, stilren och responsiv webbapplikation som hämtar och presenterar aktuell väderinformation på ett intuitivt sätt. Applikationen kombinerar geografiska data med väderdata via OpenWeather API och använder Leaflet för att skapa en interaktiv kartvisning. Projektet syftar även till att visa behärskning av tekniker som JavaScript, SCSS, CSS-animeringar samt en automatiserad arbetsprocess med Node.js och Vite. Genom detta skapas en mashup som demonstrerar både teknisk integrering och användbarhet.

1.3 *Avgränsningar*

För att projektet skulle vara hanterbart och fokuserat har vissa avgränsningar gjorts. Webbapplikationen har utformats för att vara enkel och stilren, med fokus på att visa aktuellt väder vid den tidpunkt då användaren besöker sidan. Mer avancerade funktioner, såsom väderprognoser för framtida dagar eller mer omfattande interaktivitet, har inte implementerats. Kartan är begränsad till Skåne och visar markörer för fördefinierade städer, vilket underlättar för användaren och tydliggör mashup-konceptet. Design och animeringar är avsedda att förgylla användarupplevelsen utan att överkomplicera implementationen.

1.4 Konkreta och verifierbara mål

Projektet syftar till att skapa en webbapplikation där användaren kan interagera med en karta över Skåne och direkt få information om väderförhållanden i olika städer. Användaren ska kunna se temperatur, väderbeskrivning och vindhastighet via färgkodade markörer och informativa popup-fönster. Applikationen ska även innehålla en sökfunktion som gör det möjligt att snabbt lokalisera en vald stad. Genom SCSS-baserad styling och CSS-animeringar, som en rörlig sol och knapp-animationer, skapas en visuell dynamik som gör applikationen mer engagerande. Projektet omfattar dessutom implementering av en automatiserad arbetsprocess med Node.js och Vite för bygg och optimering samt versionshantering med Git.

1.5 Översikt

Denna rapport är strukturerad för att ge en tydligare bild av projektets genomförande. Kapitel två presenterar den teoretiska bakgrunden och de tekniker som använts, medan kapitel tre beskriver metodvalen och de designprinciper som styr utvecklingen. Kapitel fyra redogör för konstruktionen av webbapplikationen, med fokus på integration av kartor, markörer, väderdata och animeringar. Kapitel fem sammanfattar resultatet och hur projektets mål har uppfyllts, och kapitel sex innehåller en diskussion kring lärodomar, begränsningar och möjliga förbättringar.

1.6 Författarens bidrag

Författaren har ansvarat för samtliga delar av projektet, från design och skapande av grafiskt material till implementation med HTML, CSS, SCSS och JavaScript. Allt grafiskt material, animeringar och funktionalitet är utvecklat av författaren, och inga externa resurser har använts utöver refererade API:er.

2 Teori

2.1 *Mashup och API-integration*

Begreppet "mashup" inom webbutveckling syftar på en webbapplikation som kombinerar innehåll och funktionalitet från flera olika källor eller tjänster för att skapa ett nytt och användbart resultat. Mashups involverar ofta integration av webbtjänster via API:er (Application Programming Interfaces), vilket gör det möjligt för olika applikationer att kommunicera och utbyta data över internet. Genom att kombinera flera API:er kan utvecklare skapa interaktiva tjänster som erbjuder information eller funktioner som inte är tillgängliga från en enda källa. (1, 2).

API:er tillhandahåller data i strukturerade format, ofta JSON (JavaScript Object Notation), som är lätt att bearbeta och visualisera i en webbläsare. JSON har blivit standardformatet för webbaserad dataöverföring tack vare dess lätta vikt, läsbarhet och enkla integration med JavaScript, vilket gör det möjligt att skapa dynamiska webbapplikationer utan att behöva ladda om sidan (3, 4).

2.2 *JavaScript och AJAX*

JavaScript är ett programmeringsspråk som används för att skapa dynamiskt och interaktivt innehåll på webbplatser. I detta projekt används JavaScript för att hantera användarinteraktioner, som att söka efter en stad och uppdatera kartan med aktuell väderdata. För att hämta data från API:er används AJAX (Asynchronous JavaScript and XML), vilket möjliggör asynkrona HTTP-förfrågningar. Detta innebär att sidan kan uppdatera innehållet i realtid utan att behöva laddas om, vilket skapar en applikationsliknande upplevelse för användaren (5, 6).

AJAX-kommunikationen i projektet sker via fetch-funktioner som anropar OpenWeather API för både geografiska koordinater och aktuellt väder. Dessa anrop returnerar JSON-objekt som sedan tolkas av JavaScript för att skapa dynamiska markörer på kartan och uppdatera popup-fönster med information om temperatur, väderbeskrivning och vindhastighet (7).

2.3 *Leaflet och kartvisualisering*

Leaflet är ett lättviktigt, open-source JavaScript-bibliotek för interaktiva kartor. Biblioteket stöder markörer, popups, lagerhantering och responsiva kartvyer, vilket gör det möjligt att enkelt integrera geografiska data i webbapplikationer. (8, 9).

För att förbättra användarupplevelser används Leaflets möjligheter att automatiskt anpassa kartans vy baserat på koordinater och markörer, samt att öppna popups med dynamiskt innehåll som hämtas från API:erna. Detta gör kartan interaktiv och engagerande, samtidigt som tekniken är lätt att integrera med ren JavaScript-kod.

2.4 Node.js och automatiserad arbetsprocess

Node.js är en plattform som möjliggör körning av JavaScript på server-sidan, vilket används i detta projekt för att driva den automatiserade arbetsprocessen via byggverktyget Vite. Vite används för att sammanställa, optimera och paketera projektets JavaScript- och SCSS-filer, vilket skapar en effektiv och modern utvecklingsmiljö. Node.js-baserade arbetsprocesser ger snabb återkoppling vid kodändringar och optimerar filstorlekar för publicering på webbservern (10, 11).

2.5 SCSS och CSS-animeringar

SCSS (Syntactically Awesome Stylesheets) är ett CSS-förprocessor-språk som tillåter användning av variabler, mixins och nesting för att skriva mer modulär och återanvändbar CSS. SCSS används i projektet för att skapa en konsekvent grafisk profil, definiera färger, typografi och layout, samt för att hantera animationer och visuella effekter (33).

CSS-animeringar används för att förbättra användarupplevelsen genom visuella effekter. Dessa animationer är lätta att implementera med CSS och har tydligt syfte, vilket är att ge visuell feedback till användaren och göra applikationen mer engagerande (12, 13).

2.6 Sammanfattning

Den teoretiska grunden för detta projekt bygger på integration av flera tekniker och verktyg för att skapa en interaktiv mashup-applikation. Mashup-konceptet möjliggör kombination av kartdata och väderinformation via API:er, medan JavaScript och AJAX används för att hantera dynamik och asynkrona dataflöden. Leaflet ger en flexibel och interaktiv kartkomponent, Node.js och Vite skapar en effektiv arbetsprocess, och SCSS med CSS-animeringar bidrar till en grafiskt tilltalande och användarvänlig presentation. Genom att kombinera dessa tekniker uppfyller projektet de krav som ställts i uppgiftsbeskrivningen och skapar en funktionell och visuellt engagerande webbapplikation.

3 Metod

Projektet syftade till att utveckla en webbaserad väderapplikation för Skåne, där aktuell väderdata visualiseras på en interaktiv karta. Fokus låg på att skapa en tydlig, stilren och användarvänlig design som samtidigt demonstrerar de tekniker som ingår i kursens krav, inklusive API-integrationer, JavaScript-baserad dynamik och CSS/SCSS-animeringar. För att undvika onödigt komplexitet hölls funktionaliteten enkel och begränsad till att visa aktuell väderinformation vid det tillfälle webbplatsen används.

3.1 *API-integration och mashup*

För projektet användes två API:er från OpenWeather: Geocoding API för att översätta stadsnamn till koordinater och Weather API för att hämta aktuell väderdata. API:erna kommunicerade genom att koordinater som hämtades från Geocoding API användes som input till Weather API, vilket är ett exempel på en mashup där data från flera webbtjänster kombineras för att skapa ny funktionalitet (14).

Innan integration testades API:erna med Thunder Client i Visual Studio Code för att säkerställa korrekt dataretur och hantering av felaktiga eller saknade platser. Förfrågningar till API:erna genomfördes asynkront med `async/await`, och felhantering implementerades för att ge användaren tydlig feedback om data inte kunde hämtas (20, 6).

3.2 *JavaScript och interaktivitet*

All dynamik i applikationen implementerades med ren JavaScript (vanilla JS) och dokumenterades med JSDoc för att implementera en användbar dokumenterad kod (15). Kartfunktionen i webbapplikationen implementerades med biblioteket Leaflet. Leaflet är ett open-source JavaScript-bibliotek för interaktiva kartor och installeras som ett npm-paket i projektet (8). Med Leaflet skapades en interaktiv karta där markörer dynamiskt genereras baserat på hämtad väderdata. Markörernas färg indikerar temperaturintervall, och varje markör har en popup med väderinformation, inklusive ikon, temperatur, väderbeskrivning och vindhastighet. Event listeners hanterar formulärinmatning för sökningar, och alla markörer uppdateras dynamiskt utan att sidan behöver laddas om (9).

För att förbättra användarupplevelsen implementerades en visuell feedback med en spinner som visas under API-anrop och döljs när datan är färdigladdad. Funktionen för att ta bort tidigare markörer vid ny sökning säkerställer att kartan alltid visar aktuell information utan förvirrande överlappande element (21).

3.3 Styling med SCSS

All styling skedde med SCSS, vilket möjliggjorde modulärt och återanvändbart CSS. Färgschema och typografi valdes för att skapa en enkel, stilren och lättläst design, med kontraster som gör informationen tydlig och visuellt tilltalande. Animeringar med CSS användes för att exempelvis visa hover-effekter på knappar, vilket bidrar till en mer engagerande användarupplevelse (12, 13).

3.4 Arbetsprocess och versionshantering

Projektet genomfördes med en automatiserad arbetsprocess baserad på Node.js och Vite (10,11). Under utveckling användes kommandot `npm run dev` för lokal utveckling med live-reload, medan `npm run build` genererade optimerade produktionsfiler för publicering.

Källkoden hanterades med Git, där arbetet organiserades i flera brancher: en `develop`-branch för pågående utveckling, en `test`-branch för testning av funktionalitet och en `main`-branch för slutlig publicering. Denna arbetsprocess säkerställde versionskontroll, effektiv felsökning och en strukturerad implementering av nya funktioner (22).

3.5 Implementation av mashup

Mashup-konceptet i projektet består i att kombinera data från Geocoding API och Weather API för att visualisera väderinformation på Leaflet-kartan. Denna sammanslagning av flera webbtjänster gör det möjligt att på ett överskådligt sätt ge användaren ett dynamiskt verktyg för att se aktuell väderdata på kartan, vilket är kärnan i projektets funktionalitet.

3.6 Testning och validering

Funktionen testades löpande under utveckling genom lokala testkörningar och verifiering av data via API-anrop. Eventuella fel i koordinat- eller väderdata hanterades med tydliga felmeddelanden till användaren. Genom denna metod säkerställdes att applikationen uppfyller kravet på en fungerande mashup som visar relevant information utan tekniska hinder.

4 Konstruktion

Webbapplikationen "Skånes Väderkarta" är byggd för att kombinera väderdata från OpenWeather med en interaktiv karta över Skåne implementerad med Leaflet. Projektets konstruktion fokuserar på att skapa en användarvänlig webbapplikation som hämtar och visar relevant information från flera JSON-baserade API:er, utan att använda frontend-ramverk, samtidigt som arbetsflödet automatiseras med Node.js och Git.

Kartfunktionen i webbapplikationen implementeras med biblioteket Leaflet. Genom att använda Leaflet kunde marker placeras på kartan dynamiskt baserat på koordinater hämtade från OpenWeather Geocoding API. Kartan initieras och konfigureras enligt kodexempel 1, där en grundvy över Skåne sätts upp.

Kodexempel 1: Initiering av Leaflet-karta

```
const map = L.map('mapContainer').setView((55.6050, 13.0038),
9);

L.tileLayer('https://{s}.tile.openstreet-
map.org/{z}/{x}/{y}.png', {

    attribution: '&copy; OpenStreetMap contributors'

}).addTo(map);
```

Kartan har zoom- och panoramafunktioner som förbättrar användarens möjlighet att interagera med webbplatsens innehåll (9).

För att hämta data från webbtjänster används JavaScript och AJAX, vilket möjliggör asynkrona anrop utan att ladda om sidan. OpenWeather API används för både geokodning (koordinater för städer) och aktuellt väder. Hämtning av koordinater sker via en asynkron funktion som använder Fetch API, vilket visas i kodexempel 2 (7, 23).

För att hämta data från webbtjänster används JavaScript och AJAX, vilket möjliggör asynkrona anrop utan att ladda om sidan. OpenWeather API används för både geokodning (koordinater för städer) och aktuellt väder. Hämtning av koordinater sker via en asynkron funktion som använder Fetch API, vilket visas i kodexempel 2 (7, 23).

Kodexempel 2: Hämtning av koordinater via API

```
async function fetchCoordinates(city) {  
  
    const response = await fetch(`https://api.openweather-  
map.org/geo/1.0/direct?q=${city}&limit=1&appid=${apiKey}`);  
  
    const data = await response.json();  
  
    return { lat: data(0).lat, lon: data(0).lon };  
  
}
```

På motsvarande sätt hämtas aktuell väderdata via en separat funktion, vilket illustreras i kodexempel 3 (7).

Kodexempel 3: Hämtning av väderdata

```
async function fetchWeather(city) {  
  
    const response = await fetch(`https://api.openweather-  
map.org/data/2.5/weather?q=${city}&appid=${api-  
Key}&units=metric&lang=sv`);  
  
    const data = await response.json();  
  
    return data;  
  
}
```

Koordinater hämtas först via Geocoding API och används sedan tillsammans med väderdata för att skapa markörer på kartan. Själva mashup-logiken, där två API:er kombineras i samma flöde, framgår i kodexempel 4 (7).

Kodexempel 4: Kombination av API-anrop (mashup)

```
const coords = await fetchCoordinates(place);  
  
const weather = await fetchWeather(place);
```

Denna process visar tydligt principen bakom en mashup, där flera tjänster kombineras för att skapa en ny funktionalitet (1).

När data har hämtats skapas en visuell representation på kartan i form av markörer. Markörerna anpassas dynamiskt baserat på temperatur och kompletteras med popup-information (24, 25).

Kodexempel 5: Skapande av markör med dynamiskt innehåll

```
const popupContent = `  
  <strong>${city}</strong><br>  
  Temp: ${weatherData.main.temp} °C<br>  
  Väder: ${weatherData.weather(0).description}  
`;  
  
L.marker((lat, lon)).addTo(map)  
  
  .bindPopup(popupContent);
```

Användargränssnittet uppdateras dynamiskt med JavaScript, vilket innebär att sökningar efter städer sker utan att sidan laddas om. Interaktionen mellan användare och system hanteras genom event listeners, vilket illustreras i kodexempel (26, 27).

Kodexempel 6: Eventhantering vid sökning

```
document.getElementById("searchForm").addEventListener("submit",  
  async (e) => {  
  
    e.preventDefault();  
  
  });
```

Denna metod säkerställer att webbapplikationen upplevs som en modern webbapp (5,6).

tylingen av webbplatsen implementerades med SASS/SCSS, vilket möjliggör modular och lättunderhållen CSS. SCSS är en vidareutveckling av CSS som tillåter användning av variabler, nesting, mixins och funktioner, vilket effektiviserar utvecklingsprocessen och minskar redundans i koden (17,24). Eftersom webbläsare inte kan tolka SCSS direkt, kompileras koden till vanlig CSS via byggverktyget Vite, vilket integreras i projektets Node.js-baserade arbetsprocess (10, 17).

Ett centralt koncept i SCSS är användningen av variabler, vilket möjliggör återanvändning av värden som färger och storlekar. I projektet definierades exempelvis färgvariabler som sedan användes genom hela layouten (28).

Kodexempel 7: Variabler

```
$background-light: #f5f5f5;

$card-bg: #ffffff;

$text-color: #333;

body {

    background: $background-light;

    color: $text-color;

}
```

Genom detta tillvägagångssätt kan förändringar i designen göras på ett centralt ställe och automatiskt slå igenom i hela webbapplikationen, vilket är en av de största fördelarna med SCSS (17, 28).

Vidare användes nesting för att strukturera CSS-regler på ett mer intuitivt sätt. Istället för att repetera selektorer kan relaterade regler kapslas in i varandra, vilket tydligt syns i formulärdesignen (29, 30).

```
.searchWrapper {  
  h2 {  
    text-align: center;  
  }  
  
  #searchForm {  
    input#placeInput {  
      &:focus {  
        border-color: variables.$accent-start;  
      }  
    }  
  }  
}
```

Denna struktur speglar HTML-hierarkin och gör koden mer lättläst, samtidigt som redundans minskar (17,18).

Mixins användes för att återanvända kodblock, särskilt för responsiv design. Ett exempel från projektet är en mixin för mobila enheter (31).

Kodexempel 9: Mixin

```
@mixin respond-mobile {  
  @media (max-width: 768px) {  
    @content;  
  }  
}  
  
@include respond-mobile {  
  #mapContainer {  
    margin: 5px auto;  
  }  
}
```

Detta gör att samma responsiva beteende enkelt kan appliceras på flera delar av webbplatsen utan att duplicera kod (17).

Utöver SCSS användes även CSS-animeringar för att förbättra användarupplevelsen. CSS-animeringar gör det möjligt att skapa visuella effekter utan JavaScript, där webbläsaren själv optimerar rendering och prestanda (12,19).

Dessa bygger på så kallade keyframes, där olika tillstånd definieras över tid. Exempelvis i form av dekorativa animationer implementerades, såsom en rörlig sol som använder translate via keyframes (32).

Kodexempel 10: Keyframe animering

```
@keyframes moveSun {  
  0% {  
    left: -50px;  
    top: 20px;  
  }  
  50% {  
    left: 50%;  
    top: 10px;  
  }  
  100% {  
    left: 100%;  
    top: 20px;  
  }  
}  
  
.sun {  
  animation: moveSun 20s linear infinite;  
}
```


För att hantera projektets filer och automatisera arbetsprocessen användes Node.js tillsammans med Vite. NPM-skript som `npm run dev` och `npm run build` användes för att köra utvecklingsserver och bygga produktionsversioner, vilket effektiviserade testning och publicering (10, 11).

Versionshantering med Git organiserades i tre huvudgrenar: `develop` för utveckling, `test` för kvalitetskontroll, och `main` för slutlig publicering. Denna arbetsprocess säkerställde att ändringar kunde testas och integreras utan att riskera stabiliteten i produktionsmiljön (16).

För att säkerställa korrekt data innan implementation testades API:erna först med verktyget Thunder Client i Visual Studio Code, vilket möjliggjorde snabb verifiering av JSON-responsens struktur och innehåll. Detta underlättade senare integration med JavaScript och Leaflet (2, 8, 20).

Sammanfattningsvis visar konstruktionen hur en mashup kan byggas genom att kombinera flera JSON-baserade API:er med dynamiska JavaScript-funktioner, SCSS-baserad styling och interaktiva kartkomponenter, samtidigt som en modern arbetsprocess med Node.js och Git säkerställer effektiv utveckling och publicering.

5 Resultat

Resultatet av projektet är en fullt fungerande webbapplikation i form av en mashup, där aktuell väderdata presenteras visuellt på en interaktiv karta över Skåne. Applikationen är publicerad via GitHub Pages och kan därmed testas i en produktionsmiljö.

OpenWeather Geocoding API används för att omvandla användarens inmatade plats till geografiska koordinater, vilka därefter används för att hämta aktuell väderdata från OpenWeather Weather API. Denna integration resulterar i att korrekt väderinformation kan presenteras för den valda platsen direkt på kartan.

Vid sidladdning renderas automatiskt markörer för sex större städer i Skåne, vilket ger användaren en möjlighet till direkt överblick över aktuellt väder i regionen utan att behöva göra en sökning. Användaren kan därefter interagera med applikationen genom att söka efter en plats inom Skåne, varpå kartan uppdateras dynamiskt. Vid en sökning tas tidigare markörer bort och en ny markör skapas baserat på den aktuella platsen, vilket säkerställer att informationen som visas alltid är relevant och aktuell.

Kartfunktionen, implementerad med Leaflet, möjliggör zoomning och panorering, vilket förbättrar användarupplevelsen och gör det enkelt att navigera mellan olika geografiska områden. Varje markör innehåller en popup med detaljerad väderinformation, inklusive temperatur, väderbeskrivning, vindhastighet samt en ikon som representerar aktuella väderförhållanden. Markörernas färg varierar beroende på temperatur, vilket ger en visuell indikation av väderläget.

Webbapplikationen är byggd med ren JavaScript och använder AJAX-anrop för att hämta data utan att ladda om sidan, vilket ger ett applikationslikt beteende. JavaScript-koden är dokumenterad med JSDoc, vilket uppfyller projektets krav på dokumentation och bidrar till ökad läsbarhet och underhållbarhet.

Styling och layout har implementerats med SCSS, vilket resulterar i en konsekvent grafisk profil med tydlig struktur och god läsbarhet. Webbplatsen är responsiv och fungerar på både desktop och mobila enheter. För att förbättra användarupplevelsen har visuella element som CSS-animationer implementerats, exempelvis i form av en laddningsindikator samt animerade knappar.

Den automatiserade arbetsprocessen med Node.js och Vite har möjliggjort effektiv utveckling och optimering av webbplatsens filer. Versionshantering har genomförts med Git, där arbetet strukturerats i olika brancher för utveckling, testning och publicering. Detta har resulterat i en stabil och välorganiserad kodbas.

Webbapplikationen har testats med verktyget Lighthouse, där den uppnår höga värden inom samtliga områden: 93 i prestanda, 98 i tillgänglighet, 96 i best practices och 91 i SEO. Dessa resultat indikerar att webbplatsen är optimerad, följer god praxis och erbjuder en god användarupplevelse.

6 Slutsatser

Vilken uppgift. Jag har kastats mellan hopp och förtvivlan ett antal för många gånger under utvecklingsprocessen av min simpla sida. Framför allt då den sidan jag hade tänkt konstruera inte gick att skapa, då jag slarvigt nog misstolkade informationen jag fick från Thunder Client när jag testade TicketMasters API.

Från början var det nämligen tänkt att jag skulle skapa en sida med namnet "Live Map Skåne", som ämnade att visa alla spelningar inom Skåne-regionen. Till en början var det svårare än det borde varit att få tag i en API-nyckel till TicketMasters Discovery API, då jag fick kontakta deras support två gånger innan jag ens fick tillträde till att logga in på deras webbplats.

Sedan avläste jag som sagt API:t alldeles för slarvigt och insåg halvvägs in i projektet att API:t inte hade tillräckligt med data gällande event som skedde utanför Malmöregionen. Jag letade då efter alternativa API:er, men de enda jag fann av intresse kostade pengar att använda, vilket jag som fattig student dessvärre inte hade något rimligt kapital att bidra med för denna uppgift.

Så jag laborerade ett tag med att omvandla sidan till "Live Map Stockholm" eller "Live Map Malmö", men istället i någon form av mental kollaps valde jag då att byta riktning och skapa en väder-webbplats. Detta för att fokusera på att faktiskt klara av uppgiftens kriterier. Av andra personliga anledningar - som jag givetvis inte skyller på, men ändå väljer att kryptiskt nämna - så har denna uppgift varit tyngre att genomföra än den borde varit.

Därav blev absolut största fokus på att just klara av att skapa en mashup-webbplats. Så därav får jag fortsätta med min "Live Map Skåne" någon gång i framtiden när jag har mer än 23 kronor på kontot (okej exakt så illa är det inte, men men..).

Men trots alla former av motgångar så är det något med att få saker att lossna som är så extremt tillfredställande och beroende framkallande. Det är märkligt när man går och lägger sig helt förstörd och håglös efter en hel dags knappande och testande av olika funktioner och koder, och ändå nästa morgon känner att man vill tillbaka till skärmen för att man måste lösa pusslet man har framför sig. Så nu till det som faktiskt gick vägen;

Som tur var så hade jag ju en stabil grund att utgå ifrån när jag valde att omvandla mitt projekt. Så att istället använda mig utav OpenWeather API för väderdata och geokodning, tillsammans med Leaflet resulterade i en fungerande och tydlig webbapplikation där användaren snabbt kan få aktuell väderinformation presenterad geografiskt, istället för event information.

Även den tekniska strukturen i projektet fungerade väl. Uppdelningen av JavaScript-kod i tydliga funktioner, användningen av `async/await` för API-

anrop samt dokumentationen med JSDoc skapade till en mer lättläst och underhållbar kodbas. Den automatiserade arbetsprocessen med Node.js och Vite fungerade som avsett för snabb testning och byggsteg som optimerade webbplatsen inför publicering. Versionshanteringen med Git och uppdelningen i branches bidrog till en mer strukturerad och lätt bearbetlig utvecklingsprocess.

Designmässigt uppnåddes målet om en enkel och stilren webbplats. Genom att medvetet undvika överkomplex funktionalitet kunde fokus läggas på kärnfunktionerna i applikationen, vilket också var i linje med projektets ändamål. SCSS användes effektivt för att strukturera stilmallar och skapa en enhetlig grafisk profil, medan enklare CSS-animationer bidrog till att ge gränssnittet ett mer levande intryck utan att påverka prestandan negativt.

Samtidigt finns det givetvis mycket som går att vidareutveckla med denna site. Man hade exempelvis kunnat implementera så att data för framtida prognoser skulle kunna visas. Man hade kunnat implementera datum-alternativ för att då välja när man önskar veta vad vädret är på en plats.

Min sida visar som sagt enbart aktuellt väder, vilket dock var ett medvetet val, då fokus låg på att uppfylla projektets krav och säkerställa att mashupen fungerade korrekt och därav skapa en mer "rakt på sak"-site som fokuserar på dagligt väder inom ett begränsat område. Siten skulle därav kunna vara en del av lokala kommuners webbplatser eller liknande.

Andra förslag på vidareutveckling av webbplatsen skulle kunna vara att inkludera fler API:er för att skapa en mer avancerad och informationsrik applikation. Även användargränssnittet skulle givetvis kunna utvecklas vidare med mer avancerade interaktioner och visualiseringar.

Sammanfattningsvis har projektet, trots sina utmaningar och omvägar, resulterat i en fungerande webbapplikation. Arbetet har bidragit till en fördjupad förståelse för hur olika webbtjänster kan kombineras, hur en modern utvecklingsprocess kan struktureras samt hur teori kan omsättas i praktisk implementation. Och framför allt hur viktigt det är att vara extremt kritiskt och noggrann under det initiala planeringsarbetet. Detta är något jag verkligen tar med mig för kommande projekt.

Jag förstår ju dock givetvis att om jag hade fått ett specifikt uppdrag av en arbetsgivare att "utveckla en event-sida som visar alla konserter inom denna region" hade det varit svårt att komma med argumentet "Absolut! Fast jag gjorde en väderapp istället. Varsågod!"... Men efter som det i detta fall var jag själv som var "arbetsgivare" så gick detta att kringgå. Denna gång.

Källförteckning

- [1] Wikipedia, "*Mashup (web application hybrid)*",
[https://en.wikipedia.org/wiki/Mashup_\(web_application_hybrid\)](https://en.wikipedia.org/wiki/Mashup_(web_application_hybrid))
Hämtad 2026-03-20
- [2] OpenWeather, "*API Documentation*",
<https://openweathermap.org/api>
Hämtad 2026-03-20
- [3] JSON.org, "*Introducing JSON*",
<https://www.json.org/json-en.html>
Hämtad 2026-03-20
- [4] Mozilla Developer Network, "*Working with JSON*",
<https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON>
Hämtad 2026-03-20
- [5] MDN Web Docs, "*AJAX*",
<https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX>
Hämtad 2026-03-20
- [6] W3Schools, "*AJAX Tutorial*",
https://www.w3schools.com/xml/ajax_intro.asp
Hämtad 2026-03-20
- [7] OpenWeather, "*Current Weather Data API*",
<https://openweathermap.org/current>
Hämtad 2026-03-20
- [8] Leaflet, "*Documentation*",
<https://leafletjs.com/reference.html>
Hämtad 2026-03-20
- [9] MDN Web Docs, "*Leaflet tutorials*",
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks
Hämtad 2026-03-20
- [10] Node.js, "*About Node.js*",
<https://nodejs.org/en/about/>
Hämtad 2026-03-20
- [11] Vite, "*Getting Started*",
<https://vitejs.dev/guide/>
Hämtad 2026-03-20

- [12] MDN Web Docs, “*Using CSS animations*”,
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Animations
Hämtad 2026-03-20
- [13] CSS-Tricks, “*CSS Animation for Beginners*”,
<https://css-tricks.com/almanac/properties/a/animation/>
Hämtad 2026-03-20
- [14] Wikipedia, “*Mashup (web application hybrid)*”,
[https://en.wikipedia.org/wiki/Mashup_\(web_application_hybrid\)](https://en.wikipedia.org/wiki/Mashup_(web_application_hybrid))
Hämtad 2026-03-21
- [15] JSDoc, “*JSDoc: API documentation for JavaScript*”,
<https://jsdoc.app>
Hämtad 2026-03-21
- [16] Git Documentation, “*Git Branching*”,
<https://git-scm.com/book/en/v2/Git-Branching-Branched-in-a-Nutshell>
Hämtad 2026-03-21
- [17] Sass Lang, “*Sass Guide*”,
<https://sass-lang.com/guide/>
Hämtad 2026-03-21
- [18] Sass Lang, “*Style Rules & Nesting*”,
<https://sass-lang.com/documentation/style-rules/>
Hämtad 2026-03-21
- [19] W3Schools, “*CSS Animations*”,
https://www.w3schools.com/css/css3_animations.asp
Hämtad 2026-03-21
- [20] Thunder Client Docs, “*Welcome – Thunder Client Documentation*”,
<https://docs.thunderclient.com/>
Hämtad 2026-03-21
- [21] Mozilla Developer Network, “*Using the Fetch API*”,
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
Hämtad 2026-03-21
- [22] Atlassian, “*Git Branching Workflow*”,
<https://www.atlassian.com/git/tutorials/comparing-workflows>
Hämtad 2026-03-21
- [23] Mozilla Developer Network, “*Fetch API*”,
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
Hämtad 2026-03-21

- [24] IBM, “*What is a mashup?*”,
<https://www.ibm.com/topics/mashup>
Hämtad 2026-03-21
- [25] Leaflet, “*Markers*”,
<https://leafletjs.com/reference.html#marker>
Hämtad 2026-03-21
- [26] Mozilla Developer Network, “*Introduction to events*”,
https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events
Hämtad 2026-03-21
- [27] Leaflet, “*Popups*”,
<https://leafletjs.com/reference.html#popup>
Hämtad 2026-03-21
- [28] Sass, “*Variables*”,
<https://sass-lang.com/documentation/variables>
Hämtad 2026-03-21
- [29] Sass, “*Nesting*”,
<https://sass-lang.com/documentation/style-rules/nesting>
Hämtad 2026-03-21
- [30] Sass, “*Using variables*”,
<https://sass-lang.com/guide#variables>
Hämtad 2026-03-21
- [31] Sass, “*Mixins*”,
<https://sass-lang.com/documentation/at-rules/mixin>
Hämtad 2026-03-21
- [32] Mozilla Developer Network, “*Using CSS animations*”,
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Animations/Using_CSS_animations
Hämtad 2026-03-21
- [33] GeeksforGeeks, “*CSS Preprocessor SASS — variables, nesting, mixins*”,
<https://www.geeksforgeeks.org/css/css-preprocessor-sass/>
Hämtad 2026-03-21